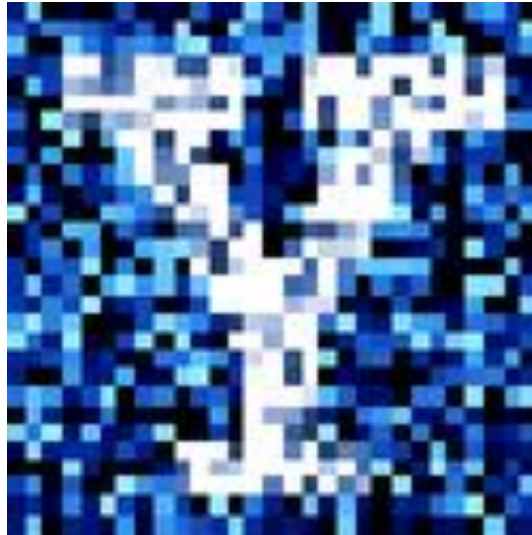


YData: Introduction to Data Science



Class 09: Pandas continued

Overview

Quick review of pandas:

- Tuples and dictionaries
- Series and DataFrames methods

Continuation of pandas:

- Calculating aggregate statistics for separate groups
- Joining DataFrames

If there is time:

- Additional practice!



Announcement: Homework 4

Homework 4 is due on Gradescope on **Sunday Feb. 15th at 11pm**

- **Be sure to mark each question on Gradescope!**



Quick review: tuples and dictionaries

Tuples are like lists but they are immutable

- `my_tuple = (10, 20, 30)` # Creating a tuple
- `my_tuple[1]` # accessing items
- `my_tuple[1] = 50` # Error! Tuples are immutable
- `val1, val2, val3 = my_tuple` # tuple unpacking



Dictionaries allow you to look up **values** based on a **key**

- `my_dict = { 'key1': 5, 'key2': 20 }`
- `my_dict['key2']`

Review: pandas Series

There are two main data structures in pandas:

- **Series**: represent one-dimensional data
- **DataFrames**: represent data tables
 - i.e., relational data



Example: egg_prices

Series are: One-dimensional ndarray with an Index

- `egg_prices.iloc[0]` # use index location
- `egg_prices.loc["1980-01-01"]` # use Index names

DATE	
1980-01-01	0.879
1980-02-01	0.774
1980-03-01	0.812

Index

values (ndarray)

Pandas Data Frame continued

Motivation: avoiding flight delays!



Pandas DataFrames

Pandas DataFrame hold
Table data

This is one of the most
useful formats to extract
insights from datasets

Often we read data into
a DataFrame using:

```
pd.read_csv("file.csv")
```

Variables

Cases

title	clean_test	binary
21 & Over	notalk	FAIL
Dredd 3D	ok	PASS
12 Years a Slave	notalk	FAIL
2 Guns	notalk	FAIL
42	men	FAIL

Selecting columns from a DataFrame

We can select a column from a DataFrame using square brackets:

```
my_df["my_col"]    # returns a Series!
```

We can select multiple columns from a DataFrame by passing a list into the square brackets

```
my_df[["col1", "col2"]]
```

Let's explore this in Jupyter!

Extracting rows from a DataFrame

We can extract rows from a DataFrame by:

1. The position they appear in the DataFrame
2. The Index values

We use the `.iloc[]` property to extract values by ***position***

```
my_df.iloc[0]
```

We use the `.loc[]` property to extract values by ***Index value***

```
my_df.loc["index_name"]
```

Extracting rows from a DataFrame

We can also extract rows through using Boolean indexing

For example:

```
bool_mask = my_df["col_name"] == 7  
my_df.loc[bool_mask]
```

Or in one step: `my_df [my_df["col_name"] == 7]`

Let's explore this in Jupyter!

Sorting rows from a DataFrame

We can sort values in a DataFrame using `.sort_values("col_name")`

- `my_df.sort_values("col_name")`

We can sort from highest to lowest by setting the argument `ascending = False`

- `my_df.sort_values("col_name", ascending = False)`

Let's explore this in Jupyter!

Adding new columns and renaming columns

We can add a column to a data frame using square brackets. For example:

```
my_df["new_col"] = values_array  
my_df["new col"] = my_df["col1"] + my_df["col2"]
```

We can rename columns by passing a dictionary to the `.rename()` method.

```
rename_dictionary = {"old_col_name": "new_col_name"}  
my_df.rename(columns = rename_dictionary )
```

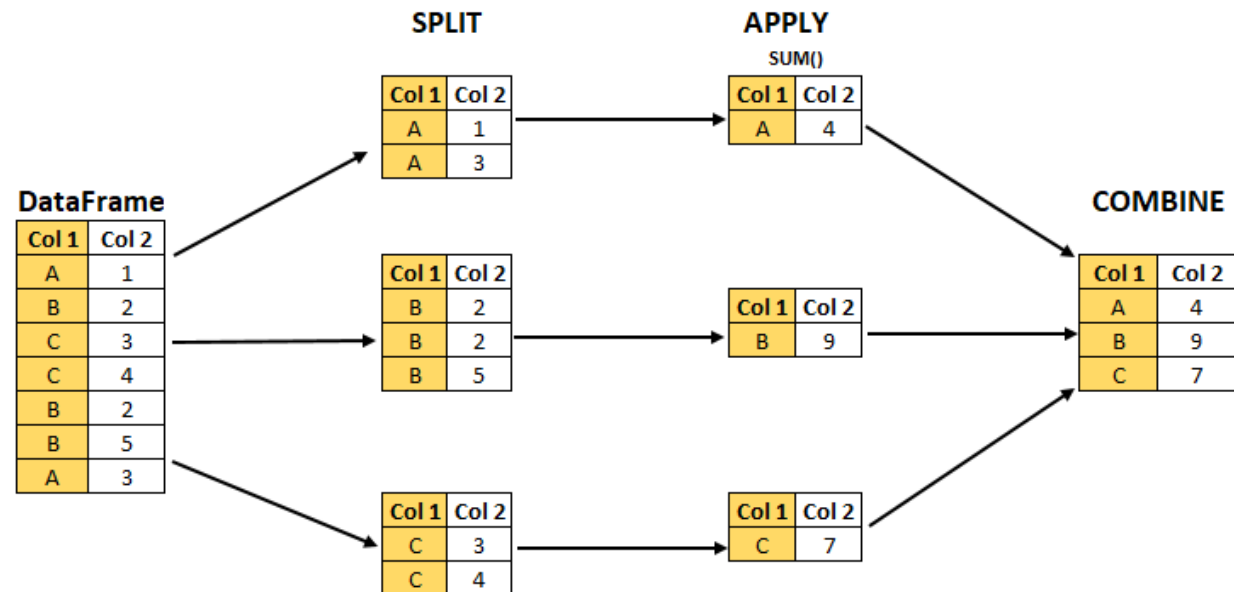
Let's explore this in Jupyter!

Creating aggregate statistics by group

We can get statistics separately by group using the `.groupby()` and `.agg()` methods

- E.g. `my_df.groupby("Col1").agg("sum")`

This implements:
“Split-apply-combine”



Creating aggregate statistics by group

There are several ways to get multiple statistics by group

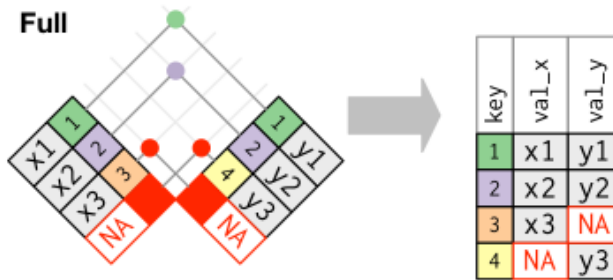
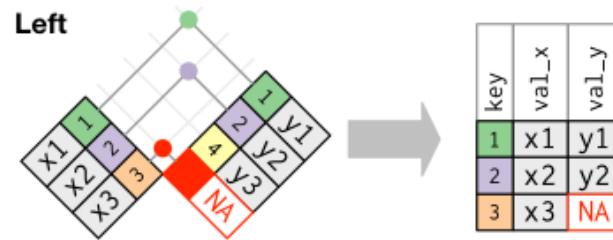
Perhaps the most useful way is to use the syntax:

```
my_df.groupby("group_col_name").agg(  
    new_col1 = ('col_name1', 'statistic_name1'),  
    new_col2 = ('col_name2', 'statistic_name2'),  
    new_col3 = ('col_name3', 'statistic_name3')  
)
```

```
nba_salaries.groupby("TEAM").agg(  
    max_salary = ("SALARY", "max"),  
    min_salary = ("SALARY", "min"),  
    first_player = ("PLAYER", "min")  
)
```

Let's explore this in Jupyter!

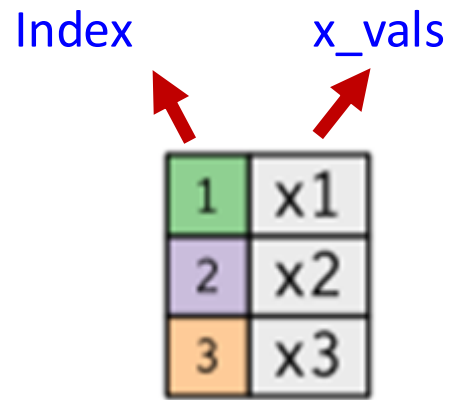
Joining data frames



Left and right tables

Suppose we have two DataFrames (or Series) called **x_df** and **y_df**

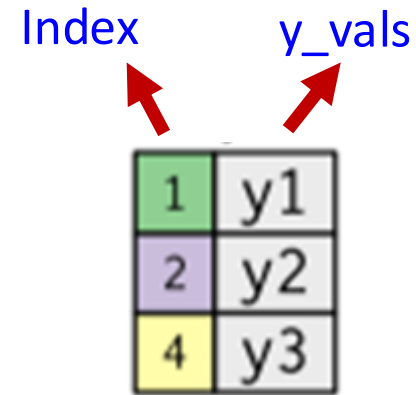
- **x_df** have one column called **x_vals**
- **y_df** has one column called **y_vals**



A diagram of a DataFrame with two columns. The first column is labeled 'Index' with a red arrow pointing to it. The second column is labeled 'x_vals' with a red arrow pointing to it. The DataFrame contains three rows: the first row has index 1 and value x1; the second row has index 2 and value x2; the third row has index 3 and value x3.

1	x1
2	x2
3	x3

DataFrame: x_df



A diagram of a DataFrame with two columns. The first column is labeled 'Index' with a red arrow pointing to it. The second column is labeled 'y_vals' with a red arrow pointing to it. The DataFrame contains three rows: the first row has index 1 and value y1; the second row has index 2 and value y2; the third row has index 4 and value y3.

1	y1
2	y2
4	y3

DataFrame: y_df

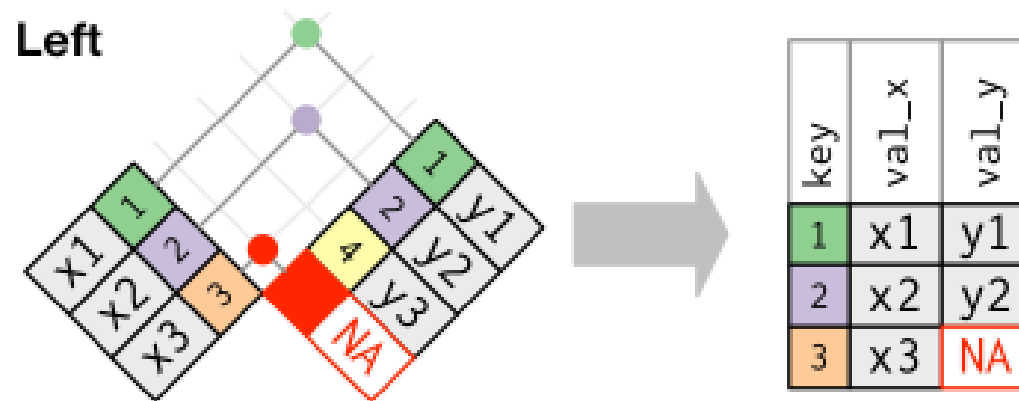
We can join these two DataFrames into a single DataFrame by aligning rows with the same Index value using the general syntax: **x_df.join(y_df)**

- i.e., the new joined data frame will have two columns: **x_vals**, and **y_vals**

Left joins

Left joins keep all rows in the left table.

Data from right table is added when there is a matching Index value, otherwise NA is added

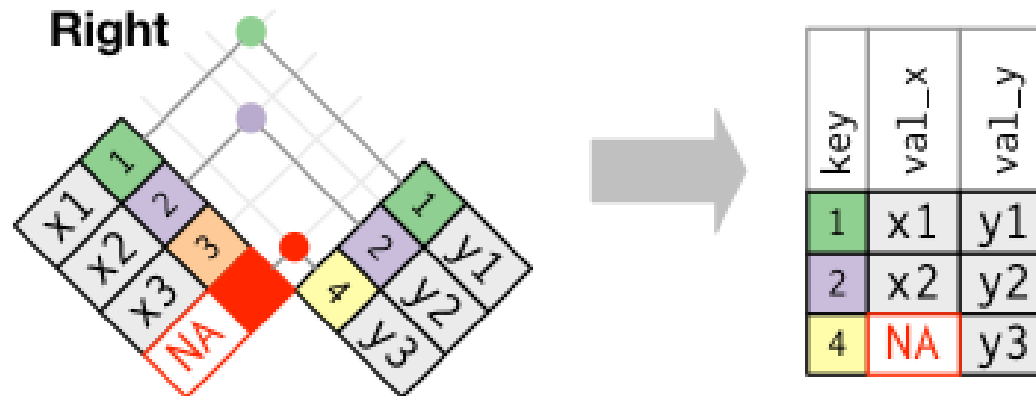


```
x_df.join(y_df, how = "left")
```

Right joins

Right joins keep all rows in the right table.

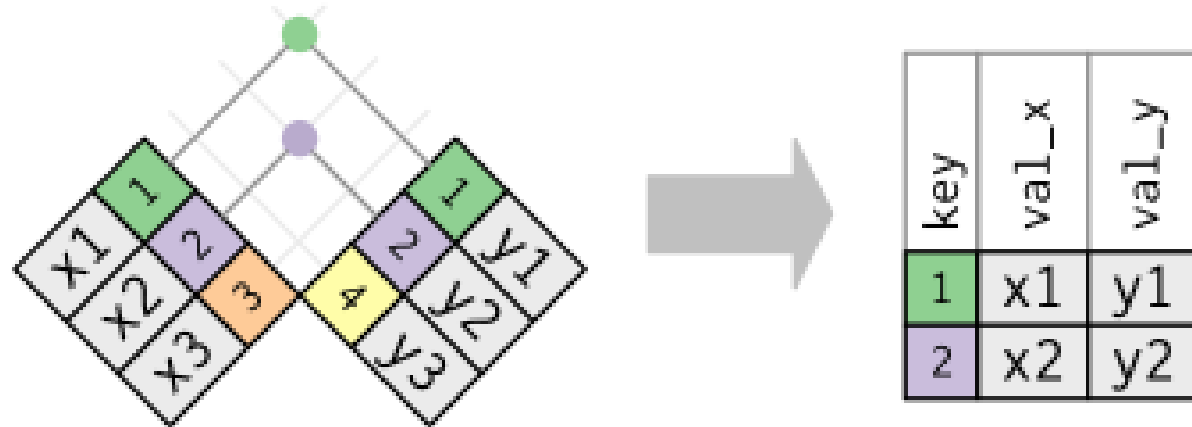
Data from left table added when there is a matching Index value otherwise NA as added



```
x_df.join(y_df, how = "right")
```

Inner joins

Inner joins only keep rows in which there are matches between the Index values in both tables.

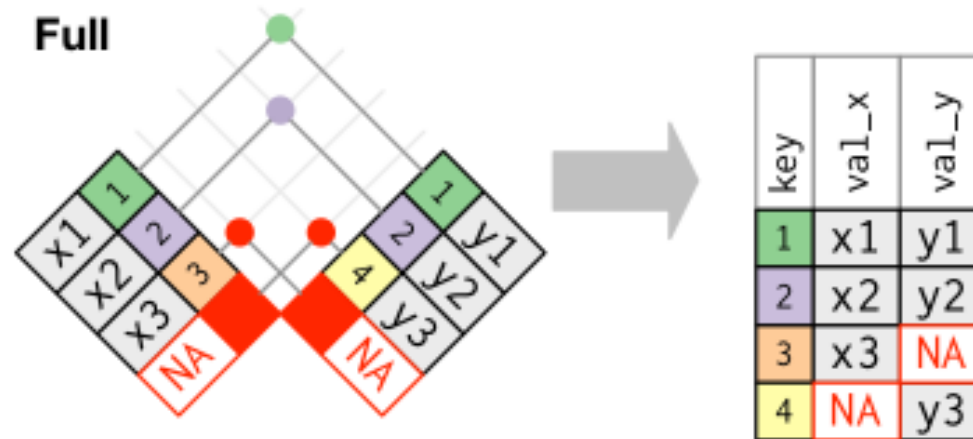


```
x_df.join(y_df, how = "inner")
```

Full (outer) joins

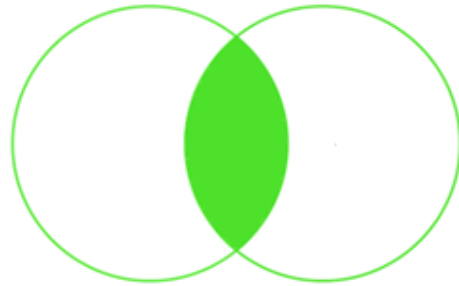
Full joins keep all rows in both table

NAs are added where there are no matches

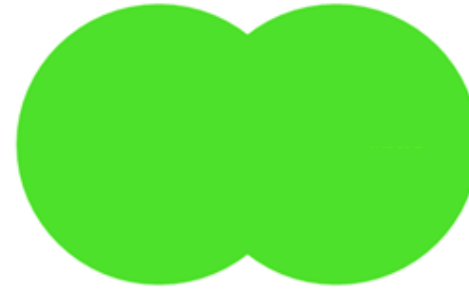


```
x_df.join(y_df, how = "outer")
```

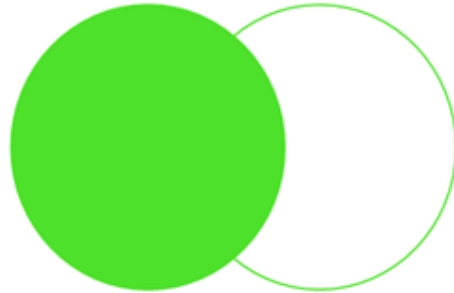
Summary



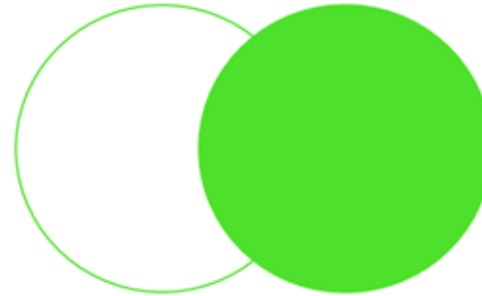
INNER JOIN



FULL OUTER JOIN



LEFT JOIN



RIGHT JOIN

“Merging” data frames

We can also join DataFrames based on values in *columns* rather than based on the DataFrames Index values

To do this we can use the merge method which has the form:

- `x_df.merge(y_df, how = "left", left_on = "x_col", right_on = "y_col")`

All the same types of joins still work

- i.e., we can do: left, right, inner and outer joins

Let's explore this in Jupyter!

Questions?



Let's do a few more practice exercises!

Work in pairs to see if you can calculate and visualize how the mean delay differs for:

1. Different hours of the day
2. Months of the year
3. Airport flight left from

If you solve these, see if you can calculate how the mean delay differs by wind speed

- You will need the *nyc23_weather.csv* to solve this